

Khó theo chiều thuận thì xét chiều ngược: bàn về ứng dụng của tư duy nghịch hướng trong giải bài

Tang Wenbin

2005

Nguồn gốc: Khó theo chiều thuận thì xét chiều ngược: bàn về ứng dụng của tư duy nghịch hướng trong giải bài

IOI China candidate team papers / 2005 / Tang Wenbin.doc

Tóm tắt

Tư duy nghịch hướng là một cách suy nghĩ về bài toán. Nó đi ngược thói quen thông thường, và chính đặc điểm ấy khiến nhiều bài toán không thể giải, hoặc rất khó giải, bằng lối suy nghĩ thuận trở nên sáng rõ. Bài viết dùng một số ví dụ để tổng kết các ứng dụng của tư duy nghịch hướng trong giải bài tin học.

Từ khóa: tư duy nghịch hướng; nguyên lý bù trừ; tìm kiếm tham số; nhị phân; quy hoạch động; ghi nhớ hóa.

1 Mở đầu

Trước hết xét một bài toán đơn giản.

Trên mặt phẳng có bốn điểm tạo thành một hình vuông cạnh 1. Ta được thực hiện thao tác sau: mỗi lần chọn hai điểm A và B , lấy điểm C đối xứng với A qua B , rồi bỏ A đi.

Hãy chứng minh rằng sau hữu hạn thao tác không thể thu được một hình vuông có cạnh lớn hơn 1.

Kết quả sau các thao tác khá phức tạp. Nếu đi theo hướng thuận, việc chứng minh mệnh đề rất khó. Ta thử nhìn từ chiều ngược lại: dễ thấy mỗi bước thao tác đều khả nghịch. Nói cách khác, nếu ta có thể làm hình vuông lớn lên, thì cũng có thể làm nó nhỏ đi.

Chứng minh. Không mất tính tổng quát, giả sử bốn đỉnh ban đầu đều là điểm nguyên. Giả sử có thể qua hữu hạn thao tác thu được một hình vuông cạnh lớn hơn 1. Khi đảo ngược toàn bộ các thao tác ấy và áp dụng lên hình vuông ban đầu, ta sẽ thu được một hình vuông cạnh nhỏ hơn 1.

Vì bốn đỉnh đều là điểm nguyên, sau mỗi thao tác tọa độ các điểm vẫn là số nguyên. Do đó hình vuông cuối cùng vừa có cạnh nhỏ hơn 1, vừa có bốn đỉnh đều là điểm nguyên. Điều này hiển nhiên không thể xảy ra. Vậy giả thiết sai, mệnh đề được chứng minh.

Ví dụ trên cho thấy ứng dụng của tư duy nghịch hướng trong toán học. Khi con đường trước mặt tưởng như bế tắc, đổi góc nhìn có thể làm bài toán mở ra ngay.

2 Ví dụ 1: Dinner Is Ready

2.1 Tóm tắt đề bài

Mẹ nấu M khúc xương để chia cho n đứa trẻ. Đứa trẻ thứ i có hai tham số Min_i và Max_i , lần lượt là số xương ít nhất và nhiều nhất mà em ấy muốn nhận.

Dữ liệu vào gồm n và M với $0 < n \leq 8$, $M > 0$, sau đó là n dòng chứa Min_i, Max_i với $0 \leq Min_i \leq Max_i \leq M$. Cần in ra số cách phân phối sao cho không lãng phí khúc xương nào.

2.2 Phân tích ban đầu

Mô hình của bài toán rất đơn giản: đếm số nghiệm nguyên của hệ

$$\begin{cases} \sum_{i=1}^n X_i = M, \\ Min_i \leq X_i \leq Max_i, \quad 1 \leq i \leq n. \end{cases}$$

Ta biết hệ đơn giản

$$\sum_{i=1}^n X_i = M, \quad X_i \geq 0$$

có số nghiệm nguyên là

$$\binom{M+n-1}{n-1}.$$

Đặt $Y_i = X_i - Min_i$. Khi đó hệ ban đầu trở thành

$$\begin{cases} \sum_{i=1}^n Y_i = M - \sum_{i=1}^n Min_i, \\ 0 \leq Y_i \leq Max_i - Min_i, \quad 1 \leq i \leq n. \end{cases}$$

Giới hạn dưới đã được xử lý bằng phép đổi biến, nhưng giới hạn trên vẫn khiến ta chưa thể tính đáp án trực tiếp.

2.3 Dùng tư duy nghịch hướng

Gọi S là tập toàn cục, tức tập các nghiệm nguyên thỏa $X_i \geq Min_i$. Gọi S_i là tập các nghiệm trong S thỏa ràng buộc $X_i \leq Max_i$, và gọi $\overline{S_i}$ là phần bù của S_i trong S , tức các nghiệm thỏa $X_i > Max_i$.

Đích cần tính là $S_1 \cap S_2 \cap \dots \cap S_n$, nhưng tập này khó đếm trực tiếp. Trái lại, các tập bù như $\overline{S_i}$ và giao của chúng lại có thể đếm được bằng cùng công thức nghiệm không âm sau khi trừ đi phần vượt quá giới hạn trên. Vì vậy ta chuyển phép tính mục tiêu sang các tập bù:

$$\begin{aligned} Answer &= |S_1 \cap S_2 \cap \dots \cap S_n| \\ &= |S| - \sum_i |\overline{S_i}| + \sum_{i < j} |\overline{S_i} \cap \overline{S_j}| - \dots + (-1)^n |\overline{S_1} \cap \overline{S_2} \cap \dots \cap \overline{S_n}|. \end{aligned}$$

Đây chính là một dạng của nguyên lý bù trừ. Đến đây bài toán đã được giải: ta không đếm trực tiếp tập đáp án khó xử lý, mà đếm các tập bù dễ xử lý hơn. Thuật toán dựa trên bù trừ có độ phức tạp

$$\mathcal{O}(2^n(n+M)).$$

3 Ví dụ 2: Greedy Path

3.1 Tóm tắt đề bài

Có n thành phố và m tuyến đường có hướng giữa chúng. Khi một du khách đi từ thành phố i đến thành phố j , chi phí phải trả là $C_{i,j}$ và thời gian đi là $T_{i,j}$. Công ty du lịch chỉ có một xe, nên giám đốc muốn tổ chức một hành trình vòng để thu được nhiều tiền nhất.

Với một chu trình G , hàm tham lam $F(G)$ bằng tổng chi phí chia cho tổng thời gian. Hãy tìm một chu trình làm $F(G)$ lớn nhất. Nếu không có chu trình thì in 0; nếu có nhiều đáp án, in tùy ý một chu trình tối ưu.

3.2 Phân tích ban đầu

Bài toán yêu cầu tìm một chu trình, nhưng mục tiêu không phải là tổng trọng số lớn nhất hoặc nhỏ nhất, nên không thể áp dụng trực tiếp các thuật toán kinh điển.

Gọi đồ thị là $G = (V, E)$, và gọi S là tập mọi chu trình $C = (V', E')$ của G . Ta cần tìm một chu trình trong S làm

$$F(C) = \frac{\sum_{e \in E'} C_e}{\sum_{e \in E'} T_e}$$

đạt giá trị lớn nhất.

3.3 Dùng tư duy nghịch hướng

Nếu đã biết $C^* = (V^*, E^*) \in S$ là chu trình tối ưu, thì

$$t^* = F(C^*) = \frac{\sum_{e \in E^*} C_e}{\sum_{e \in E^*} T_e}.$$

Từ đó ta định nghĩa

$$o(t) = \max_{C=(V', E') \in S} \left(\sum_{e \in E'} C_e - t \sum_{e \in E'} T_e \right).$$

Ta đưa ra phỏng đoán: nếu $o(t^*) = 0$, và tồn tại $C^* = (V^*, E^*) \in S$ sao cho

$$\sum_{e \in E^*} C_e - t^* \sum_{e \in E^*} T_e = 0,$$

thì C^* là một chu trình tối ưu.

Chứng minh. Giả sử tồn tại một chu trình khác $C_1 = (V_1, E_1) \in S$ tốt hơn C^* . Khi đó

$$\frac{\sum_{e \in E_1} C_e}{\sum_{e \in E_1} T_e} > t^*,$$

suy ra

$$0 < \sum_{e \in E_1} C_e - t^* \sum_{e \in E_1} T_e \leq o(t^*).$$

Điều này mâu thuẫn với $o(t^*) = 0$. Vậy C^* là chu trình tối ưu.

Nếu t^* là đáp án tối ưu, ta có tính chất

$$\begin{cases} o(t) = 0 \iff t = t^*, \\ o(t) < 0 \iff t > t^*, \\ o(t) > 0 \iff t < t^*. \end{cases} \quad (1)$$

Từ đây thuật toán xuất hiện tự nhiên. Bắt đầu với một khoảng chứa t^* , chẳng hạn

$$t_l = \min_{e \in E} \frac{C_e}{T_e}, \quad t_h = \max_{e \in E} \frac{C_e}{T_e}.$$

Mỗi lần lấy trung điểm t của t_l và t_h , rồi tính $o(t)$.

Cách tính $o(t)$ như sau: với mỗi cạnh $e \in E$, đặt trọng số mới

$$W_e = C_e - tT_e.$$

Dùng Floyd để tìm chu trình có tổng trọng số lớn nhất trong đồ thị có hướng. Giá trị lớn nhất ấy chính là $o(t)$.

Theo tính chất (1), ta cập nhật t_l hoặc t_h , tiếp tục nhị phân cho đến khi đạt độ chính xác yêu cầu. Nếu số lần nhị phân là K , độ phức tạp thời gian là $\mathcal{O}(Kn^3)$. Tuy đây không phải một thuật toán đa thức nghiêm ngặt, với giới hạn của đề bài nó vẫn tìm được đáp án rất nhanh.

4 Ví dụ 3: Building Towers

4.1 Tóm tắt đề bài

Có N khối lập phương, cần dùng chúng để xây tháp. Mỗi tháp có H tầng; tầng dưới cùng chứa M khối. Với mỗi tầng phía trên, số khối ở tầng đó phải đúng bằng số khối ở tầng ngay dưới cộng 1 hoặc trừ 1.

Trong tài liệu gốc có một hình minh họa cho một tháp hợp lệ với $H = 6$, $M = 2$, $N = 13$. Nhiệm vụ là tính số tháp khác nhau có thể xây được; lưu ý không bắt buộc dùng hết mọi khối. Hai tháp được xem là khác nhau nếu tồn tại một tầng i mà số khối của tầng thứ i trong hai tháp khác nhau. Mỗi tháp được biểu diễn bằng một dãy H số nguyên dương từ đáy lên đỉnh. Ngoài ra cần trả lời các truy vấn: in tháp có thứ tự từ điển thứ K .

Dữ liệu vào gồm N, H, M với $N \leq 32767$, $H \leq 60$, $M \leq 10$, sau đó là các truy vấn K ; dòng cuối cùng chứa -1 . Dữ liệu ra gồm tổng số tháp khác nhau và, với mỗi truy vấn, dãy biểu diễn tháp tương ứng.

4.2 Phân tích ban đầu

Khi vừa nhìn đề, theo quán tính ta rất dễ nghĩ đến quy hoạch động cổ điển. Gọi $F[i, j, k]$ là số phương án khi tầng hiện tại có i khối, còn j tầng và còn có thể dùng k khối. Khi đó có thể viết công thức

$$F[i, j, k] = F[i + 1, j + 1, k - i] + F[i - 1, j + 1, k - i].$$

Điều kiện biên là $F[M, H, N] = 1$, các trạng thái còn lại ban đầu bằng 0.

Dễ thấy quy hoạch động này liên quan tới khoảng $N \cdot M \cdot K$ trạng thái, trường hợp lớn nhất vào khoảng $60 \cdot 70 \cdot 2400 \approx 10^7$. Nếu mỗi trạng thái dùng một số double, bộ nhớ cần ít nhất khoảng 80 MB. Hơn nữa, do cần in tháp thứ K theo thứ tự từ điển, ta không thể dùng tối ưu cuộn chiếu. Cả thời gian lẫn bộ nhớ đều khó chấp nhận, nên hướng quy hoạch động thuận bị bế tắc.

4.3 Dừng tư duy nghịch hướng

Quy hoạch động từ dưới lên gây áp lực lớn về thời gian và không gian. Ta thử “đảo chiều” xử lý và xem xét tìm kiếm từ trên xuống.

Tìm kiếm thường bị xem là đồng nghĩa với kém hiệu quả và hàm mũ. Nhưng nếu thêm ghi nhớ hóa, nó lại trở thành một quá trình quy hoạch động nghịch hướng. Đảo thứ tự quy hoạch có hai lợi ích.

Thứ nhất, nhìn bài toán từ trên xuống giúp ta có cái nhìn tổng thể hơn, thuận lợi cho việc quyết định tìm kiếm thế nào, tìm gì trước, tìm gì sau và cắt tỉa ra sao. Trong bài này, tìm kiếm sẽ không đi vào nhiều trạng thái vô lý, chẳng hạn số khối rõ ràng không đủ hoặc sai khác về tính chẵn lẻ. Với các bài tối ưu, tìm kiếm có ghi nhớ cũng phối hợp tốt với cắt tỉa theo tính tối ưu, tránh nhiều trạng thái không có giá trị, điều mà quy hoạch động thuận khó làm.

Thứ hai, trong các bài đếm tương tự bài này, có thể dùng ghi nhớ hóa một phần: chỉ lưu những trạng thái dễ bị tìm lại nhiều lần. Như vậy số trạng thái phải lưu giảm xuống và tốc độ tra cứu tăng lên.

Khi cài đặt, dùng bộ ba (N, H, M) để biểu diễn một trạng thái: tầng hiện tại có M khối, còn H tầng, và còn có thể dùng N khối. Ký hiệu $F(N, H, M)$ là số phương án của trạng thái ấy. Ta lưu kết quả bằng bảng băm và thêm một số tối ưu sau.

- Tính trước số khối ít nhất và nhiều nhất mà trạng thái hiện tại còn cần. Nếu N nhỏ hơn nhu cầu tối thiểu thì đáp án rõ ràng là 0; nếu N lớn hơn nhu cầu tối đa, đặt lại N bằng nhu cầu tối đa để tránh lưu trùng các trạng thái tương đương.
- Nếu N không nhỏ hơn nhu cầu tối đa và $H \leq M$, thì mọi lựa chọn lên/xuống ở $H - 1$ bước còn lại đều hợp lệ, nên đáp án là 2^{H-1} .
- Để giảm lượng trạng thái lưu trữ, đặt một ngưỡng lưu: chỉ khi $F(N, H, M)$ không vượt quá ngưỡng ấy mới đưa trạng thái vào bảng băm.

Vì sao tối ưu cuối cùng có hiệu quả? Cây trạng thái của tìm kiếm mở rộng theo dạng hàm mũ. Càng xuống sâu, số trạng thái càng nhiều và tần suất tra cứu càng cao; gần gốc, trạng thái phân bố thưa hơn, và khi $F(N, H, M)$ lớn thì tần suất truy cập chúng cũng giảm. Do đó ta có thể chấp nhận tìm lại các trạng thái lớn ấy để giảm bộ nhớ và tăng tốc tra cứu bằng băm.

Hiệu quả thực nghiệm của “quy hoạch động nghịch hướng” như sau.

N	H	M	Số trạng thái của QHD thuận	Số trạng thái lưu trong bảng băm	Tỉ lệ
1000	40	10	1200000	3055	392.80
1500	50	10	2625000	5371	488.74
1200	60	10	2880000	49875	57.74
1500	60	10	3600000	38018	94.69

Ngay trong trường hợp xấu nhất, quy hoạch động nghịch hướng cũng chỉ cần xử lý khoảng một phần năm mươi số trạng thái. Đến đây, bài toán đã được giải trọn vẹn.

5 Tổng kết

Ví dụ thứ nhất là tư tưởng chuyển sang phân bù: khi không biết xử lý tập ban đầu thế nào, ta đếm phần bù để nhìn bài toán từ mặt ngược lại. Ví dụ thứ hai không nhìn mặt ngược của bài toán, mà nhìn mặt ngược của một sự thật: hiện ta chưa biết đáp án, vậy nếu đã biết đáp án thì điều gì sẽ xảy ra? Ví dụ thứ ba đảo chiều thứ tự quy hoạch khi quy hoạch động cổ điển không giải nổi, từ đó thu được quy hoạch động nghịch hướng, tránh nhiều trạng thái vô hiệu và cải thiện mạnh cả thời gian lẫn không gian.

Ba ví dụ trên chỉ là một vài ứng dụng đơn giản của tư duy nghịch hướng. Tinh thần “khó thuận thì xét ngược” giúp ta phá vỡ quán tính tư duy, lấy lùi làm tiến, tránh chỗ mạnh và đánh vào chỗ yếu của bài toán. Khi con đường quen thuộc không còn lối, đổi hướng suy nghĩ có thể đem lại lời giải rất tự nhiên.

Lời cảm ơn

Tác giả đặc biệt cảm ơn bạn Zhou Yuan từ Trường Trung học số 1 Vu Hồ, An Huy, cũng như tất cả những người đã giúp đỡ tác giả.

Tài liệu tham khảo

1. Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi đấu tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Gunnar W. Klau và các đồng tác giả, *The Fractional Prize-Collecting Steiner Tree Problem On Trees*.
3. Ren Chunong, *Bàn về tư duy nghịch hướng*.

Phụ lục về chương trình

Mã nguồn cho ví dụ 3, bài *Building Tower*, nằm trong tệp `Tower.cpp` đi kèm tài liệu gốc.

A Nguyên đề của các ví dụ

A.1 Example 1: Dinner Is Ready

Dinner is ready! WishingBone’s family rush to the table. WishingBone’s mother has prepared many delicious bones. WishingBone has several brothers, namely WishingTwoBones, WishingThreeBones. WishingBone wants at least one bone, WishingTwoBones two and WishingThreeBones three. :-P But as they are not too greedy (or they want to keep shape), they decide that they want at most twice of their minimal requirement. Now let’s suppose mother has prepared 7 bones, one way to distribute them is 2-2-3, the other two ways are 1-3-3 and 1-2-4. Of course mother doesn’t want to waste any bones, so if she had prepared 13 bones, she would end up without any feasible distribution.

As curious as WishingBone, mother wants to know how many different ways she can distribute those bones.

Input. The first line of input is a positive integer $N \leq 100$, which is the number of test cases. The first line of each case contains two integers n and m ($0 < n \leq 8$, $0 < m$), where n is the number of children and m is the number of bones mother has prepared. The next n lines have two integers $mini$ and $maxi$ ($0 \leq mini \leq maxi \leq m$), which are the minimal and maximal requirement of the i -th child.

Output. N integers which are the numbers of ways to distribute the m bones, one per line.

Sample Input.

```

2
3 7
1 2
2 4
3 6
3 13
1 2
2 4
3 6

```

Sample Output.

```

3
0

```

A.2 Example 2: Greedy Path

There are N towns and M routes between them. Recently, some new travel agency was founded for servicing tourists in these cities. We know cost which tourist has to pay, when traveling from town i to town j which equals to C_{ij} and time needed for this travel, T_{ij} . There are many tourists who want to use this agency because of very low rates for travels, but agency still has only one bus. Head of agency decided to organize one big travel route to gain maximal possible amount of money. Scientists of the company offer to find such a cyclic path G , when greedy function $f(G)$ will be maximum. Greedy function for some path is calculated as total cost of the path divided by total time of path. But nobody can find this path, and Head of the company asks you to help him in solving this problem.

Input. There are two integers N and M on the first line of input file ($3 \leq N \leq 50$). Next M lines contain routes information, one route per line. Every route description has format A, B, C_{ab}, T_{ab} , where A is starting town for route, B is ending town for route, C_{ab} is cost of route and T_{ab} is time of route ($1 \leq C_{ab} \leq 100$, $1 \leq T_{ab} \leq 100$, $A \neq B$). Note that order of towns in route is significant: route (i, j) is not equal to route (j, i) . There is at most one route in one direction between any two towns.

Output. You must output requested path G in the following format. On the first line of output file you must output K , the number of towns in the path ($2 \leq K \leq 50$); on the second line, numbers of these towns in order of passing them. If there are many such ways, output any one of them; if there are no such ways, output "0".

Sample Input.

```

4 5
1 2 5 1
2 3 3 5
3 4 1 1
4 1 5 2
2 4 1 10

```

Sample Output.

```
4
1 2 3 4
```

A.3 Example 3: Building Towers

Time Limit: 2 second. Memory Limit: 1000 KB.

There are N cubes in a toy box which has 1-unit height; the width is double the height. The teacher organizes a tower-building game. The tower is built by the cubes. The height of the tower is H levels. The bottom of the tower contains M cubes; and for all above level, each must contain a number of cubes which is exactly 1 less than or greater than the number of cubes of the level right below it.

The original document shows an example of a tower with $H = 6$, $M = 2$, $N = 13$.

Your task is to determine how many different towers can be there. Two towers are considered different if there is at least one number i ($1 < i \leq H$) so that the i -th level of one tower contains a different number of cubes to the i -th level of the other tower. Each tower is represented by an array of H positive integers which determines the structure of the tower from the bottom level up to the top level. Those arrays are sorted ascendingly.

Input. The first line contains three positive numbers N, H, M ($N \leq 32767$, $H \leq 60$, $M \leq 10$). Each following line, except the last, contains an integer K . The last line contains number -1 .

Output. The first line is the total of different towers that can be founded. Each following line contains an array that represents the structure of the tower corresponding to the number K from input.

Sample Input.

```
22 5 4
1
10
-1
```

Sample Output.

```
10
4 3 2 1 2
4 5 4 5 4
```

Hint. Numbers which are on the same line should be separated by at least one space. You do not have to use all N cubes.

This problem also appears as Zhejiang University Online Judge 1442, Saratov State University Online Judge 236, and Ural Online Judge 1148. One related note in the source says that this is a classical combinatorics problem that can be solved by the divider principle.